Quarter-Car Suspension Modeling and Simulation

In this project, you will use MATLAB and Simscape Multibody to build and tune a quarter-car suspension model using an automated road test suite. You'll first develop the model using Simscape Multibody to represent the mechanical structure and motion of a vehicle corner (body corner + wheel assembly connected through suspension and tire compliance). Then, you'll create a test suite in MATLAB of road profiles that automatically computes objective metrics for comfort and road holding. Finally, you'll tune the suspension parameters to meet constraints and improve performance across all road cases.

Use this Live Script as a template to get started on your project solution. You may include helper functions at the bottom of this document or as separate files. For a tutorial on how to use Live Scripts, check out this video.

Suggested Tasks

1. Build a baseline quarter-car multibody model in Simscape
2. Create a road test suite of 3-5 road profiles
3. Log signals and define objective metrics
4. Build an automated test runner
5. Tune suspension parameters
6. Validate robustness with a simple variation

Table of Contents

Break Down the Problem

In the space below, define the problem statement:

Design and tune a quarter-car suspension model to balance ride comfort and road holding across a variety of realistic roads, using a model-based design in Simscape Multibody.

List the requirements, constraints, and success criteria for the project:

The model must represent sprung mass, unsprung mass, suspension spring, and tire compliance in the simscape multibody. The suspension must be evaluated across 4-5 road profiles. Suspension travel and tire deflection has to stay within defined packaging limits. Comfort should be minimized subject to those constraints, and the tuned design has to remain robust under +/- 10% component tolerance variation in Ks and Cs across a 20-trial Monte Carlo test.

What is your proposed solution?

List the steps you will need to take to achieve your proposed solution:

What challenges do you face in the process of achieving the solution?

Task 1: Build a baseline quarter-car multibody model

In Simscape Multibody,

Build two rigid bodies:

- Sprung mass (vehicle body corner)
- Unsprung mass (wheel assembly)

Connect them with:

- Suspension spring + damper (parameterized)
- Tire vertical compliance element (parameterized or fixed)

Provide a road displacement input (e.g., a vertically moving “road plate” or equivalent road excitation subsystem)

Tip: Keep geometry simple (vertical DOF-focused) so the model is stable and fast to simulate.

```
% Open Simulink to build your vehicle model in Simscape Multibody
% Optionally, provide code below to open your .slx model and run the
% simulation

setupQuarterCar;
assignin('base', 'Ks', Ks);
assignin('base', 'Cs', Cs);
assignin('base', 'Kt', Kt);
assignin('base', 'Ct', Ct);
assignin('base', 'Ms', Ms);
assignin('base', 'Mu', Mu);

open_system('quarterCarModel');
simOut = sim('quarterCarModel', 'ReturnWorkspaceOutputs', 'on');
```

Task 2: Create a road test suite (3-5 cases)

Implement a set of road displacement profiles suclear;

such as:

- Speed bump (smooth hump)
- Pothole (smooth dip)

- Rough road (band-limited noise)
- (optional) Washboards (sinusoidal corrugation)
- (optional) Two bumps (repeatability / transient recovery)

Below are a few options for building the road suite (choose one approach or mix them):

MATLAB-generated signals (recommended):

- Create each road profile as a `timeseries` or `[t, z]` array in `roadSuite.m`
- Examples: speed bump (half-sine hump over a fixed duration), pothole (negative half-sine dip), rough road (filtered noise using `randn` + a simple filter)
- Feed into the model using From Workspace (or Signal Editor) blocks.

Simulink blocks (no MATLAB scripting required):

- speed bump / pothole: Signal Builder or Signal Editor with a hand-drawn profile
- washboard: Sine Wave block
- two bumps: Pulse Generator (smoothed with a filter) or concatenated segments in Signal Editor
- rough road: Band-Limited White Noise block (optionally followed by a Transfer Fcn / Discrete Filter block to shape the spectrum)

Data-driven (optional):

- Import a recorded profile from CSV (e.g., `readtable`) and feed it via From Workspace.
- Deliverable: `roadSuite.m` (or `roadSuite.mat`) that produces named road inputs, plus a short note explaining how each profile was made.

```
% Road profiles are generated by roadSuite.m, a function that produces
% a timeseries for each named road case:
%
%   - speedBump: smooth 5 cm hump (half-sine), 1.0-1.5 s
%   - pothole: smooth 5 cm dip (negative half-sine), 1.0-1.5 s
%   - roughRoad: band-limited random noise (~1 cm amplitude), generated
%     with randn and filtered using a 2nd-order Butterworth low-pass
%     filter (10 Hz cutoff) so the road doesn't jump instantaneously
%   - washboard: sinusoidal corrugation (~2 cm amplitude, 4 Hz)
%   - twoBumps: two speed bumps in sequence, testing suspension
%     recovery between events
%
% Each profile is fed into the Simscape Multibody model via a
% "From Workspace" block, using the variable roadInput.

roadCase = 'speedBump'; % change to test other road cases
roadInput = roadSuite(roadCase);
```

Task 3: Log signals and define objective metrics

Log at minimum:

- Sprung-mass vertical acceleration
- Suspension travel (relative displacement)
- Tire deflection (relative wheel-to-road displacement) or normal-force proxy

Compute metrics per road case, such as:

- Comfort metric: RMS sprung acceleration (and/or peak)
- Packaging metric: max suspension travel
- Road-holding metric: max tire deflection (or variability)

Deliverable: a function like:

- `results = scoreSuspension(simout, roadName)`
- returning a struct/table of metrics and a single "score"

```
% Insert your code here (or make use of helper functions or additional .m
or .mlx files and indicate where
% they can be found). Ensure your code is well-documented.

% scoreSuspension.m is implemented as a separate function file in the
% same project folder as this Live Script. It logs sprung-mass
% acceleration, suspension travel, and tire deflection from the sim
% output, computes RMS comfort, max travel, and max tire deflection
% metrics, and returns a struct with those metrics plus a combined score.

setupQuarterCar;
simIn = Simulink.SimulationInput('quarterCarModel');
out = sim(simIn);
results = scoreSuspension(out, roadCase)
```

```
results =
struct with fields:
          roadName: 'twoBumps'
         comfortRMS: 0.0071
       packagingMax: 1.9253e-04
    roadHoldingMax: 0.0501
             score: 0.0574
```

Task 4: Build an automated test runner

Create a script/function that:

- Runs all road cases automatically
- Logs results to a table
- Outputs a clear pass/fail for each requirement
- Generates a single summary figure (or a brief report)

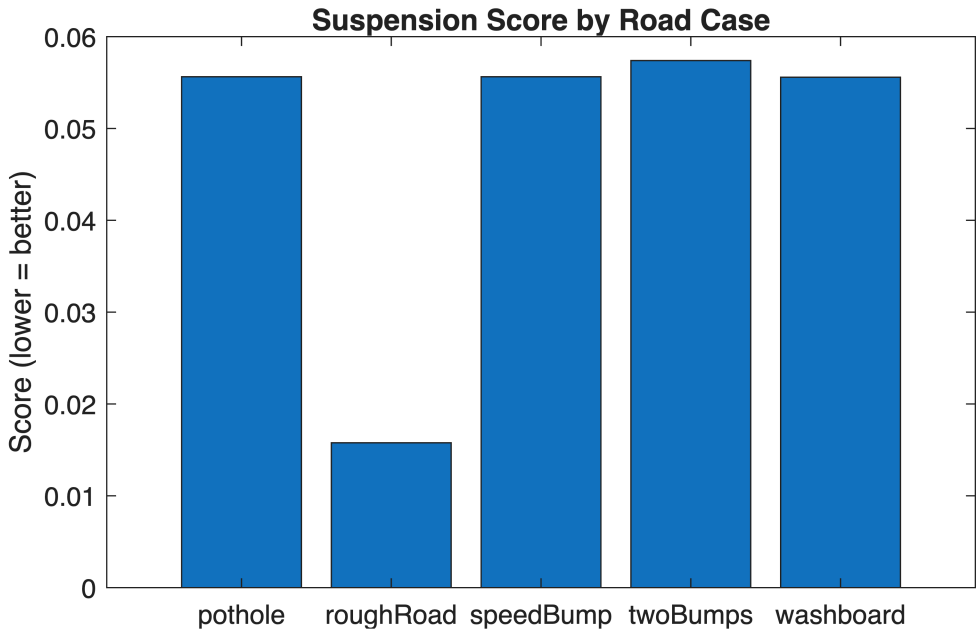
Example deliverable: `runAllTests.m` → returns `summaryTable` and saves plots to `/results`

```
% Here, it is recommended to create a separate runAllTests.m file
% (or include as a local helper function) that combines the work you've
done
% so far here and additionally performs the steps outlined in the task
description.
% You can run it from here.
% runAllTests.m is implemented as a separate function file in the same
% project folder as this Live Script. It loops through all 5 road cases,
% runs a simulation for each, scores it with scoreSuspension.m, checks
% pass/fail against comfort/travel/tire-deflection limits, and saves a
% summary table + bar chart to a "results" folder.

summaryTable = runAllTests()
```

summaryTable = 5×6 table

	roadName	comfortRMS	packagingMax	roadHoldingMax	score	pass
1	'speedBump'	0.0054	1.9253e-04	0.0501	0.0556	false
2	'pothole'	0.0054	1.9253e-04	0.0501	0.0556	false
3	'roughRoad'	0.0104	6.5996e-05	0.0053	0.0158	true
4	'washboard'	0.0350	4.4594e-04	0.0201	0.0556	true
5	'twoBumps'	0.0071	1.9253e-04	0.0501	0.0574	false



summaryTable = 5×6 table

	roadName	comfortRMS	packagingMax	roadHoldingMax	score	pass
1	'speedBump'	0.0054	1.9253e-04	0.0501	0.0556	false
2	'pothole'	0.0054	1.9253e-04	0.0501	0.0556	false
3	'roughRoad'	0.0104	6.5996e-05	0.0053	0.0158	true
4	'washboard'	0.0350	4.4594e-04	0.0201	0.0556	true

	roadName	comfortRMS	packagingMax	roadHoldingMax	score	pass
5	'twoBumps'	0.0071	1.9253e-04	0.0501	0.0574	false

Task 5: Tune suspension parameters

Tune at least:

- Suspension spring stiffness `Ks`
- Suspension damping `Cs`

Two tuning options:

Option A — Parameter sweep (recommended for beginners)

- Sweep `Ks` and `Cs` across small ranges
- Score each design across all road cases
- Select a design that meets constraints and minimizes score

You can implement the parameter sweep in MATLAB using `Simulink.SimulationInput` objects to programmatically create simulation runs with different values of suspension stiffness (Ks) and damping (Cs) without modifying the model manually.

Implement the parameter sweep in MATLAB for each design point, configure the parameter values in a `SimulationInput` object, execute the sweep using `parsim` (see here for more information on Running Parallel Simulation) when Parallel Computing Toolbox is available, or `sim` as a serial fallback.

To reduce repeated runtime, use Fast Restart for interactive scripted simulations, and optionally Rapid Accelerator for faster batch execution of the design space.

Option B — Lightweight optimization (recommended for intermediate-level users)

- Use a simple search (pattern search / constrained minimization / custom heuristic)
- Minimize comfort subject to travel/deflection limits

Deliverable: chosen parameter values + justification using plots/tables.

```
% Insert your code here that will output the optimal values for suspension
% spring stiffness (Ks) and suspension damping (Cs) based on your
% simulation. Your output should also include plots/tables that justify the
% optimal parameter values.

tuneResults = tuneSuspension()
```

Ks	Cs	worstScore	allPass
12000	1000	0.057395	false
12000	1500	0.058366	false

```
    12000    2000     0.059407      false
    12000    2500     0.060413      false
    12000    3000     0.061388      false
    14000    1000     0.057505      false
    14000    1500     0.058448      false
    14000    2000     0.059451      false
    14000    2500     0.060451      false
    14000    3000     0.061401      false
    16000    1000     0.058751      false
    16000    1500     0.058495      false
    16000    2000     0.059498      false
    16000    2500     0.060452      false
    16000    3000     0.061441      false
    18000    1000     0.060201      false
    18000    1500     0.058531      false
    18000    2000     0.059544      false
    18000    2500     0.060526      false
    18000    3000     0.061434      false
    20000    1000      0.06149      false
    20000    1500     0.058617      false
    20000    2000     0.059589      false
    20000    2500     0.060547      false
    20000    3000     0.061469      false


--- Best combination(lowest worst-case score) ---
     Ks       Cs     worstScore    allPass
    _____    ____    __________    _______

    12000    1000     0.057395      false
tuneResults = 25×4 table
```

	Ks	Cs	worstScore	allPass
1	12000	1000	0.0574	false
2	12000	1500	0.0584	false
3	12000	2000	0.0594	false
4	12000	2500	0.0604	false
5	12000	3000	0.0614	false
6	14000	1000	0.0575	false
7	14000	1500	0.0584	false
8	14000	2000	0.0595	false
9	14000	2500	0.0605	false
10	14000	3000	0.0614	false
11	16000	1000	0.0588	false
12	16000	1500	0.0585	false
13	16000	2000	0.0595	false
14	16000	2500	0.0605	false
15	16000	3000	0.0614	false
16	18000	1000	0.0602	false
17	18000	1500	0.0585	false

	Ks	Cs	worstScore	allPass
18	18000	2000	0.0595	false
19	18000	2500	0.0605	false
20	18000	3000	0.0614	false
21	20000	1000	0.0615	false
22	20000	1500	0.0586	false
23	20000	2000	0.0596	false
24	20000	2500	0.0605	false
25	20000	3000	0.0615	false

Task 6: Validate robustness with one simple variation

Pick one low-effort robustness test:

- Payload change: increase sprung mass by +25% and rerun the suite
- Component tolerance: apply ±10% variation to `Ks` and `Cs` for a small Monte Carlo set (e.g., 20 trials)

Deliverable: a robustness summary (worst-case metrics and pass rate).

```
% Insert code here to change the parameters as suggested and rerun your
% test suite. Your code should return a robustness summary that includes
% worst-case metrics and pass rate.

robustResults = robustnessTest()
```

```
      roadName        comfortRMS    packagingMax    roadHoldingMax     score      pass
    ______________    __________    ____________    ______________    ________    _____

    {'speedBump'}     0.0053659      0.00019253       0.050074       0.055632    false
    {'pothole'  }     0.0053659      0.00019253       0.050074       0.055632    false
    {'roughRoad'}       0.01028       5.733e-05      0.0036582       0.013996    true
    {'washboard'}      0.034991      0.00044594       0.020144       0.055581    true
    {'twoBumps' }     0.0071281      0.00019253       0.050074       0.057395    false

Pass rate: 40% (2/5 road cases passed)
robustResults = 5×6 table
```

	roadName	comfortRMS	packagingMax	roadHoldingMax	score	pass
1	'speedBump'	0.0054	1.9253e-04	0.0501	0.0556	false
2	'pothole'	0.0054	1.9253e-04	0.0501	0.0556	false
3	'roughRoad'	0.0103	5.7330e-05	0.0037	0.0140	true
4	'washboard'	0.0350	4.4594e-04	0.0201	0.0556	true
5	'twoBumps'	0.0071	1.9253e-04	0.0501	0.0574	false

Interpretation of Results

Summarize your findings by interpreting their physical/engineering meaning: Our results show that the suspension design's performance is dominated by tire deflection rather than sprung-mass comfort. Across all 25 Ks/Cs combinations tested, comfort and suspension travel stayed within the limits, but tire deflection was consistently exceeding the 0.03 m requirement on impulsive road events, but passing on continuous inputs.

What are some limitations of your work? The tuned design doesn't achieve allPass = true on any combination, since tire deflection is governed by Kt/Ct, which were outside this task's tuning scope. The robustness test also only evaluated one scenario, which didn't include the alternative 10% tolerance monte Carlo study.

What are practical next steps? Practical next steps is to extend the sweep to include Kt/Ct as tunable parameters, since tire compliance appears to be the primary driver of the road-holding failures. Running the 10% Ks/Cs Monte Carlo study would also give a fuller picture of robustness to manufacturing variation.